
Spell-Aware Word Prediction Using N-gram and Transformer Language Models

Jonathan Tadesse
jtadesse@kth.se

1 Introduction

Word prediction is a common feature in modern text input systems, especially on mobile keyboards. As a user types, the system presents a small set of word completions or next-word suggestions. If the intended word appears early enough, the user can select it directly and save keystrokes. At the core of this process is a language model that estimates which word is likely to appear next given the current context. A useful word prediction system must be accurate, efficient, and robust to typing errors. It should update suggestions after each keystroke, rank likely words highly, and still recover when the typed prefix contains mistakes. For example, a user who types *pthon* may intend to write *python*, so the system should not rely only on exact prefix matching.

In this project, we implement word prediction using two language models: an interpolated N-gram model and a Transformer with a byte-pair encoding (BPE) tokeniser [1], [2]. We also add a spell-correction module that expands the candidate set to words within a small Damerau-Levenshtein edit distance, rather than only considering words with the exact typed prefix [3], [4]. A web-based GUI connects the components and allows the user to switch between models, datasets, and spell-correction strategies. We evaluate the system on three datasets with different text characteristics: TinyStories, WikiText-2, and Mobile SMS. This allows us to compare how the models behave on simple narrative text, encyclopedic text, and short informal messages and to study how domain differences affect both clean word prediction and spell-aware prediction.

2 Related Work

N-gram language models are a traditional approach to word prediction. They estimate the next-word probability from the previous $n - 1$ words using counts from a training corpus. Since high-order N-grams can be sparse, interpolation is often used to combine different context lengths. Jelinek-Mercer smoothing [5] mixes unigram, bigram, trigram, and higher-order probabilities so unreliable long-context estimates can fall back on shorter contexts. More advanced methods such as Kneser-Ney smoothing [6] can improve estimates further, but here we use a simpler interpolated model with dataset-specific tuned weights.

Transformer language models [2] have become dominant in many language modelling tasks. Unlike N-gram models, they can use longer context and learn more flexible text representations. However, most Transformer models use subword tokens rather than whole words, which adds a step for word prediction: token-level probabilities must be converted into whole-word candidate scores. In our implementation, we score each candidate word using the probability of its first BPE token, following the course assignment approach.

Spell correction is commonly based on edit distance, especially Levenshtein distance and the Damerau extension, which also allows adjacent transpositions [3], [4]. These methods measure how many edits transform a typed string into a candidate word. Traditional noisy-channel approaches combine edit-based similarity with a language-model score to rank corrections [7]. Our method follows the same idea, but uses a simplified scoring function that works with both the N-gram and Transformer models.

3 Method

3.1 Datasets

We trained and evaluated on three datasets chosen to cover different text domains: simple narrative text, encyclopedic text, and short informal messages. This allows us to test whether the models generalise across different writing styles, vocabulary sizes, and sentence lengths.

- **TinyStories** [8]: A collection of short children’s stories written in simple English. The language is repetitive and sentence structures are relatively simple, making it useful for testing prediction in a story-like domain.
- **WikiText-2** [9]: A standard language modelling benchmark extracted from Wikipedia. It contains more varied sentence structures, many rare words, and proper nouns, making it a harder prediction task.
- **Mobile SMS** [10]: A corpus of text messages. The messages are short and often formulaic, but the vocabulary is large because of abbreviations, informal spelling, and user-specific expressions.

3.2 Dataset preprocessing

The datasets were preprocessed differently for the N-gram model and the Transformer, because the models use different input representations. For the N-gram model, all text was lowercased and punctuation was removed. TinyStories was additionally split into individual cleaned sentences, since the N-gram model predicts words from local word contexts.

For the Transformer model, the datasets were not split into individual sentences. Instead, one training example was kept per line. For TinyStories, each line corresponds to a full story; for WikiText-2, each line corresponds to a non-empty line or paragraph from the original corpus; and for Mobile SMS, each line corresponds to one text message. We did this to preserve more of the original context for the Transformer while still giving the BPE tokeniser clean text to learn from. Table 1 shows the final dataset sizes.

Table 1: Dataset sizes after preprocessing for the N-gram and Transformer models.

(a) N-gram format				(b) Transformer format			
Dataset	N-gram train	N-gram validation	N-gram test	Dataset	Transformer train	Transformer validation	Transformer test
TinyStories	15,603,516 sentences	193,167 sentences	3,706,169 sentences	TinyStories	799,943 stories	9,999 stories	189,987 stories
WikiText-2	85,757 sentences	9,046 sentences	10,434 sentences	WikiText-2	17,556 lines	1,841 lines	2,185 lines
Mobile SMS	4,754,043 messages	377,955 messages	237,182 messages	Mobile SMS	4,754,043 messages	377,955 messages	237,182 messages

3.3 N-gram Language Model

Before training the N-gram model, the text was cleaned into a simple word-level form: all text was lowercased, punctuation and special characters were removed or replaced by spaces, and repeated whitespace was collapsed. This was to reduce vocabulary size and ensure that words such as *word*, *word*, and *Word* were treated as the same token.

We then built an interpolated 4-gram model with Laplace (add-1) smoothing [11]; see Equation 4 in the Appendix for more details. The vocabulary was built from the training split with minimum count `min_count = 2`, so every word observed at least twice in training was kept. Words not seen in the training vocabulary were mapped to the unknown-word token `<UNK>`.

Each sentence was padded with three start symbols, `<s>` and one end-of-sentence symbol, `</s>`, to allow the 4-gram model to use up to three previous words as context. For example,

the dog runs $\rightarrow$ `<s> <s> <s> the dog runs </s>`.

During training, we stored counts for unigrams, bigrams, trigrams, and 4-grams.

The interpolated 4-gram probability of a word w given context c is then given by:

$$P_{\text{interp}}(w \mid c) = \lambda_1 P_1(w) + \lambda_2 P_2(w \mid c_1) + \lambda_3 P_3(w \mid c_{1:2}) + \lambda_4 P_4(w \mid c_{1:3}) \quad (1)$$

where $P_1, \dots, P_4$ are the Laplace-smoothed unigram through 4-gram probabilities and the weights satisfy $\sum_i \lambda_i = 1$. The weights were tuned separately for each dataset by grid search on the validation set, maximizing the saved-keystroke ratio at top-3.

3.4 Transformer Language Model

We trained a decoder-only Transformer [2] based on the Assignment 3 architecture. The final models use 4 Transformer blocks, 4 attention heads, embedding dimension 256, dropout 0.1, pre-layer normalisation, residual connections, and causal self-attention. TinyStories and WikiText-2 have about 5.85M trainable parameters, while Mobile SMS has about 7.29M because it uses a larger BPE vocabulary.

A separate BPE tokeniser was trained from scratch for each dataset. TinyStories and WikiText-2 use 5,000 subword tokens, while Mobile SMS uses 8,000 to better cover its larger and more irregular vocabulary. The models were trained with AdamW [12], learning rate 5×10^{-4} and weight decay 10^{-6} on a CUDA GPU. TinyStories and WikiText-2 use block size 512 and batch size 8; Mobile SMS uses block size 128 and batch size 50, based on the Mobile SMS grid search. Training was configured for 3 epochs and evaluation used the best validation-loss checkpoint.

During training, validation loss was computed every 100 training steps on fixed-length BPE token blocks from the validation split. The checkpoint with the lowest validation loss was saved and used for later word-prediction and spell-correction evaluation.

3.5 Prefix Index and Candidate Generation

We used a prefix index to make autocomplete candidate generation efficient. For each dataset, we built a vocabulary V from the corresponding training data. We then created an index that maps each possible prefix to the vocabulary words that begin with that prefix:

$$I(p) = \{w \in V : w_{1:|p|} = p\}.$$

For a vocabulary word $w = w_1w_2 \dots w_m$, we inserted w under all of its prefixes:

$$\epsilon, w_1, w_1w_2, \dots, w_1w_2 \dots w_m,$$

where ϵ denotes the empty prefix. At prediction time, the typed prefix p was used to retrieve the candidate set $I(p)$. The language model then only ranks these candidate words instead of scoring the full vocabulary at every keystroke. This allowed the system to update suggestions at every keystroke.

3.6 Word Prediction

We evaluate word prediction by simulating autocomplete. Given a word context c and a prefix p for the current word, our system first retrieves candidate completions from the prefix index described in Section 3.5. The candidates are then ranked by either the N-gram model or the Transformer.

For the N-gram model, we score each candidate word w with the interpolated word-level probability

$$P_{\text{interp}}(w \mid c),$$

where c is the preceding word context.

For the Transformer, we first encode the context using the dataset-specific BPE tokeniser. Since the Transformer predicts subword tokens, we encode each candidate word w as " " + w and score it using the probability of its first BPE token:

$$\text{score}(w) = P(t_1(w) \mid c),$$

where $t_1(w)$ is the first BPE token of w .

During evaluation, we type each target word one character at a time. For a target word $w = w_1w_2 \dots w_m$, we query the system with

$$\epsilon, w_1, w_1w_2, \dots, w_1w_2 \dots w_m.$$

After each query, we check whether the target word appears in the top- k suggestions. If it does, we record how many characters had to be typed before the word was suggested. We then use this value to compute the saved-keystroke ratio.

3.7 Spell Correction

Normal word prediction assumes the user types correctly. Our spell corrector relaxes this: instead of only considering words that literally start with the characters typed so far, it also considers words whose *beginning* is close to what was typed under Damerau-Levenshtein edit distance [3], [4].

Concretely, suppose the user has typed p characters (possibly with typos). For each vocabulary word c , we compute the edit distance between the typed prefix p and the first $|p|$ characters of c . The four allowed edit operations are **insertion**, **deletion**, **substitution** and **transposition** of adjacent characters. If the distance is within the threshold, c is a candidate.

We implemented three strategies, each differing in the maximum distance allowed and the cost assigned to each operation:

- **S1:** Standard Damerau-Levenshtein, maximum distance 1. All four operations cost 1.0.
- **S2:** Standard Damerau-Levenshtein, maximum distance 2. All four operations cost 1.0.
- **S3:** Weighted Damerau-Levenshtein, maximum distance 2. Transpositions cost 0.5 (adjacent letter swaps are the most common slip). An optional keyboard-layout variant additionally reduces the substitution cost to 0.5 for keys that are adjacent on the QWERTY layout (Euclidean distance ≤ 1.5); this variant is available in the GUI but was not used in the reported evaluation numbers.

Candidate words within the distance threshold are ranked by the following score:

$$\text{score}(c) = \log P(c \mid \text{context}) - \lambda \cdot \frac{d(p, c_{[:|p|]})}{\text{max_dist}} + \mu \cdot \frac{\log(\text{freq}(c) + 1)}{\text{max_log_freq}} \quad (2)$$

where p is the typed prefix, $c_{[:|p|]}$ denotes the first $|p|$ characters of candidate c , $P(c \mid \text{context})$ comes from either the N-gram or the Transformer model and the last term is a small frequency boost so rare words do not crowd out common ones. The parameters λ and μ were tuned on a small held-out set of corrupted examples (see Section 4.2). The best settings were $\lambda = 1.0$ and $\mu = 0.05$ for most datasets, and $\lambda = 1.0$ and $\mu = 0.20$ for WikiText-2.

4 Evaluation

4.1 Metrics

4.1.1 Top- k accuracy

Top- k accuracy is the fraction of target words that appear somewhere in the top- k suggestions, at some point as the user types the word character by character. A hit at top-3 means the correct word was among the three suggestions shown.

4.1.2 Saved-keystroke ratio

Saved-keystroke ratio measures how much typing was actually saved. For each target word, we simulate the user typing it one character at a time and check when the correct word first appears in the top- k list. If it appears after c characters have been typed, the system saved $\text{len}(\text{word}) - c$ keystrokes. We sum these savings across all words and divide by the total number of characters that would have been typed without any prediction:

$$\text{KS-ratio} = \frac{\sum_w \max(0, \text{len}(w) - c_w)}{\sum_w \text{len}(w)} \quad (3)$$

This is a more realistic measure than accuracy alone, because it rewards the system for surfacing the correct word early, not just for eventually finding it. For spell correction, we use the same two metrics but the simulated typing follows the *corrupted* word rather than the correct one.

4.1.3 Perplexity

As an additional metric, we report **perplexity**, computed from the average negative log-likelihood; see Equation 5 in the Appendix for the formula. For the N-gram model, we computed it over words; for the Transformer, we computed it over BPE tokens.

4.2 Experimental setup and model selection

4.2.1 Transformer Architecture Selection

The 4-block, 4-head, dim-256 architecture was not chosen arbitrarily. We justify the choice separately for each dataset.

TinyStories. We compared the default model (4 blocks, 4 heads, 256 dimensions) against a small model (1 block, 1 head, 256 dimensions). Both were evaluated by generating text with the prompt “*Once upon a time*” at checkpoints of 10k, 20k and 30k iterations (temperature 0.8, top- k 40, 120 tokens). The 4-block model produced noticeably more

coherent stories: at 30k iterations it maintained a consistent character, setting, and dialogue, while the 1-block model at the same checkpoint still generated semantically unstable output. We therefore reused the default configuration for this project.

WikiText-2. We ran a dedicated grid search on WikiText-2 to confirm the architecture choice for a very different domain. Four architecture sizes were evaluated: tiny (1 block, 1 head, dim 128), small (2 blocks, 2 heads, dim 128), default (4 blocks, 4 heads, dim 256), and medium (6 blocks, 8 heads, dim 512), each combined with three stride values (8, 32, 64), giving 12 configurations in total. Each configuration was trained for 30,000 iterations on a CUDA GPU. The best validation-loss checkpoint was saved for each run. The default architecture with stride 8 achieved the lowest validation loss and was selected as the final WikiText-2 model.

Mobile SMS. The Mobile SMS grid search followed the same procedure but compared only the small and default architectures with strides 8 and 32 (4 runs, 10,000 iterations each), because the SMS corpus is much smaller. Results are shown in Table 10. The default architecture with stride 8 was again the best configuration.

4.2.2 N-gram model selection

We tuned the N-gram interpolation weights in Equation 1 on held-out validation subsets, using saved-keystroke ratio at top-3 as the selection criterion. We searched over all non-negative interpolation weights ($\lambda_1, \lambda_2, \lambda_3, \lambda_4$) that sum to one, using a step size of 0.1:

$$\lambda_i \in \{0.0, 0.1, \dots, 1.0\}, \quad \sum_{i=1}^4 \lambda_i = 1.$$

We used 5,000 validation sentences for TinyStories, 1,000 for WikiText-2, and 100 validation messages for Mobile SMS. The N-gram vocabulary size after training for each dataset can be found in Appendix 11.

4.2.3 Word prediction

We evaluated word prediction by simulating autocomplete for each target word, as described in Section 3.6. For the N-gram model, we used 7,500, 4,000, and 1,000 preprocessed test sentences for TinyStories, WikiText-2, and Mobile SMS, respectively. For the Transformer model, word prediction was evaluated on held-out test examples from the Transformer format test splits. We used smaller test sets than for the N-gram model because Transformer evaluation is more computationally expensive: at every simulated keystroke, the context must be BPE-tokenised, passed through the neural model and used to score candidate completions. The N-gram model, on the other hand, has fast count-based probability lookups. Furthermore, the preprocessing also differs: the N-gram model is evaluated on individual cleaned sentences, while the Transformer test examples are full stories for TinyStories, paragraph lines for WikiText-2 and complete SMS messages for Mobile SMS. Therefore, we reasoned that a smaller number of Transformer examples can still contain many target words.

4.2.4 Spell correction

We generated fixed corrupted test sets from the test split by corrupting the final word of eligible examples. We considered an example eligible only if it contained at least two alphabetic tokens, so that the target word had at least one preceding context word. The target word had to have length at least two. We required the corrupted word to differ from the original, have the requested Damerau-Levenshtein edit distance, have length at least two and not already occur as a valid vocabulary word. We evaluated S1 on the one-edit set, while S2 and S3 were evaluated on the two-edit set. We used 1,000 generated examples each for TinyStories and WikiText-2, and 100 generated examples for Mobile SMS. We generated the corrupted examples so that edit operations were approximately evenly distributed across all datasets and strategies; see Table 8 in the Appendix for more details. During evaluation, we skipped examples whose target word was outside the model vocabulary.

4.2.5 Spell parameter tuning

We tuned the spell-ranking parameters λ and μ in Equation 2 using corrupted validation sets generated from the validation split. The validation sets contained 100 examples each for TinyStories and WikiText-2 and 20 examples for Mobile SMS. We performed a grid search over

$$\lambda \in \{0.25, 0.5, 0.75, 1.0\}, \quad \mu \in \{0.05, 0.1, 0.2\}.$$

The parameter setting was selected using the Top-3 saved-keystroke ratio under strategy S3 as the validation criterion. For WikiText-2, the parameters were $\lambda_{\text{edit}} = 1$ and $\mu_{\text{freq}} = 0.2$. For TinyStories and Mobile SMS, the parameters were $\lambda_{\text{edit}} = 1$ and $\mu_{\text{freq}} = 0.05$. See details in Appendix Table 9.

5 Results

5.1 N-gram interpolation weights

Table 2: Best interpolation weights per dataset.

Dataset	λ_1 (unigram)	λ_2 (bigram)	λ_3 (trigram)	λ_4 (four-gram)	Number of validation examples used
TinyStories	0.0	0.0	0.1	0.9	5000
WikiText-2	0.0	0.1	0.9	0.0	1000
Mobile SMS	0.0	0.1	0.7	0.2	100

TinyStories benefits most from the 4-gram component, with the best setting $\lambda_4 = 0.9$, because its stories contain many repeated local patterns and short, predictable phrases. WikiText-2 instead relies mainly on the trigram component ($\lambda_3 = 0.9$), since its larger vocabulary and more varied syntax make exact 4-gram contexts less reliable. Mobile SMS shows a similar preference for trigram context, but keeps a small 4-gram contribution, with $\lambda_3 = 0.7$ and $\lambda_4 = 0.2$.

5.2 Word Prediction Results

Table 3: Word-prediction results on the test sets. Acc. = top- k accuracy; KS = saved-keystroke ratio; PPL = perplexity.

Dataset	Model	Top-1		Top-2		Top-3		Top-4		PPL
		Acc.	KS	Acc.	KS	Acc.	KS	Acc.	KS	
TinyStories	N-gram	32.53%	57.80%	44.10%	65.61%	51.42%	68.03%	56.47%	69.29%	258.52
TinyStories	Transformer	95.44%	71.43%	96.73%	74.72%	97.18%	76.71%	97.55%	78.00%	5.68
WikiText-2	N-gram	16.59%	40.14%	22.28%	48.88%	26.08%	52.89%	28.68%	55.30%	16357.84
WikiText-2	Transformer	89.45%	41.04%	92.84%	52.58%	93.74%	57.88%	94.03%	61.14%	53.44
Mobile SMS	N-gram	16.29%	43.84%	23.02%	53.53%	27.59%	57.59%	31.01%	60.22%	2621.21
Mobile SMS	Transformer	94.44%	51.28%	97.22%	62.72%	97.70%	67.42%	97.79%	70.36%	65.63

Table 3 shows that the Transformer obtains higher accuracy than the N-gram model for every dataset and Top- k setting. The largest Top-1 accuracy improvement is observed on Mobile SMS, increasing from 16.29% to 94.44%. The Transformer also gives substantially lower perplexity on all datasets, with the largest reduction on WikiText-2, from 16357.84 to 53.44.

5.3 Spell correction

Table 4: Spell correction on TinyStories. Acc = top- k accuracy, Saved = saved-keystroke ratio.

Strategy	Model	Top-1		Top-2		Top-3		Top-4	
		Acc	Saved	Acc	Saved	Acc	Saved	Acc	Saved
S1	N-gram	90.09%	46.12%	94.69%	54.47%	96.40%	58.41%	97.10%	60.95%
S1	Transformer	86.69%	43.96%	91.19%	51.91%	93.39%	55.43%	94.69%	57.66%
S2	N-gram	77.88%	35.00%	85.39%	44.10%	88.79%	49.16%	90.59%	52.31%
S2	Transformer	74.67%	35.50%	81.38%	43.14%	84.58%	47.04%	85.99%	49.41%
S3	N-gram	78.48%	35.10%	85.79%	44.10%	89.19%	49.14%	90.89%	52.39%
S3	Transformer	74.87%	35.56%	81.38%	43.20%	84.78%	47.10%	86.09%	49.45%

Table 5: Spell correction on WikiText-2.

Strategy	Model	Top-1		Top-2		Top-3		Top-4	
		Acc	Saved	Acc	Saved	Acc	Saved	Acc	Saved
S1	N-gram	73.28%	26.03%	81.71%	34.92%	84.70%	39.41%	85.59%	41.72%
S1	Transformer	69.51%	17.68%	78.49%	25.89%	80.71%	29.98%	82.15%	32.45%
S2	N-gram	55.99%	17.29%	65.63%	24.10%	71.06%	27.80%	72.95%	29.70%
S2	Transformer	53.10%	11.53%	62.20%	17.55%	66.41%	20.65%	69.40%	23.09%
S3	N-gram	57.32%	17.61%	66.08%	24.46%	70.84%	27.87%	72.39%	29.89%
S3	Transformer	53.10%	11.50%	62.42%	17.59%	66.30%	20.65%	69.07%	23.00%

Table 6: Spell correction on Mobile SMS

Strategy	Model	Top-1		Top-2		Top-3		Top-4	
		Acc	Saved	Acc	Saved	Acc	Saved	Acc	Saved
S1	N-gram	82.65%	27.73%	88.78%	34.01%	90.82%	37.65%	90.82%	40.69%
S1	Transformer	88.78%	32.59%	91.84%	38.87%	92.86%	44.53%	93.88%	47.77%
S2	N-gram	63.27%	13.97%	76.53%	20.24%	82.65%	26.52%	83.67%	28.95%
S2	Transformer	74.49%	21.66%	82.65%	29.76%	84.69%	33.60%	86.73%	38.87%
S3	N-gram	64.29%	14.37%	76.53%	20.85%	82.65%	26.52%	84.69%	29.15%
S3	Transformer	75.51%	21.66%	82.65%	29.96%	84.69%	34.01%	86.73%	38.87%

Top-3 Saved Keystroke Ratio Summary

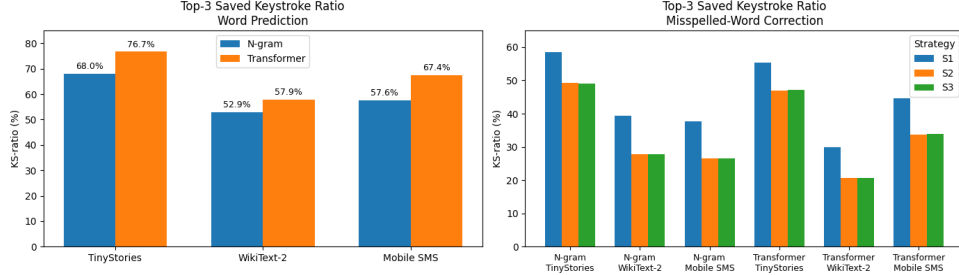


Figure 1: Top-3 SKR for clean word prediction and spell correction across all datasets, strategies, and models.

Figure 1 summarizes the Top-3 KS results. For word prediction, the Transformer improves over the N-gram model on all datasets, with the largest gain on Mobile SMS. For spell correction, strategy S1 gives the highest SKR in all model-dataset settings. The N-gram model gives higher spell-correction KS on TinyStories and WikiText-2, whereas the Transformer gives higher KS on Mobile SMS.

Table 7: Saved-keystroke ratio by context length before the corrupted word. For example, 3–5 means that the model has 3 to 5 context words before the corrupted word. This is for top- $k = 3$.

Dataset	Model	Strat.	1–2	3–5	6–10	11–20	21–50
TinyStories	N-gram	S1	56.19%	57.36%	58.01%	60.82%	59.68%
		S2	45.36%	46.30%	48.81%	53.52%	53.23%
		S3	45.10%	46.30%	48.90%	53.34%	53.23%
	Transformer	S1	37.63%	49.30%	56.53%	64.55%	66.13%
		S2	25.77%	38.61%	49.52%	56.99%	56.45%
		S3	25.77%	38.71%	49.57%	57.08%	56.45%
WikiText-2	N-gram	S1	45.33%	35.71%	35.71%	40.59%	39.26%
		S2	25.33%	22.86%	26.19%	26.81%	30.00%
		S3	25.33%	22.86%	26.34%	26.89%	30.05%
	Transformer	S1	19.33%	31.43%	29.46%	30.56%	30.00%
		S2	8.67%	17.62%	22.02%	19.48%	22.80%
		S3	8.67%	17.62%	22.02%	19.48%	22.80%
Mobile SMS	N-gram	S1	30.56%	37.84%	40.00%	32.03%	50.00%
		S2	13.89%	27.03%	28.57%	24.84%	32.14%
		S3	13.89%	27.03%	28.00%	25.49%	32.14%
	Transformer	S1	22.22%	39.19%	50.29%	37.91%	66.07%
		S2	8.33%	31.08%	38.29%	30.07%	48.21%
		S3	8.33%	32.43%	38.29%	30.72%	48.21%

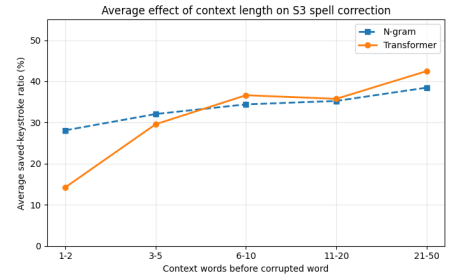


Table 7 shows that S1 is the strongest spell-correction strategy across most context-length intervals. S2 and S3 perform almost identically. The Transformer benefits more from longer context on TinyStories and Mobile SMS, while WikiText-2 shows no consistent gain from additional context. Overall, the Transformer seems to benefit more from additional context, as confirmed by the plot.

6 Discussion

Clean word prediction. The results in Table 3 show that the Transformer performs better than the N-gram model for clean word prediction across all datasets. We expected this, since the Transformer has a more informative representation

of the previous context than the N-gram model that relies on exact word-count patterns. This is confirmed by the perplexity scores: the Transformer has significantly lower perplexity than the N-gram model across all datasets. What is interesting, however, is that the saved-keystroke ratio for the Transformer model does not improve as much as the Top- k accuracy across top- $k = 1, 2, 3, 4$, as it does for the N-gram model. We reasoned that this is because the prefix index already removes many wrong candidates once the user has typed a few letters. Therefore, even if the N-gram model ranks the correct word poorly at the empty-prefix stage, the correct word may still appear after only a few typed characters. Our takeaway is that Top- k accuracy mainly measures how well the language model ranks candidates, while saved-keystroke ratio also depends on how quickly the typed prefix reduces the candidate set.

Dataset differences. The interpolation weights in Table 2 show that the best N-gram context length depends on the dataset. TinyStories favours the 4-gram component because it contains many repeated local patterns. WikiText-2 relies mainly on trigram context, since rare words, names, and varied syntax make exact 4-gram matches less reliable. Mobile SMS also favours trigram context, with a small 4-gram contribution, because the messages are short and formulaic but have a noisy and irregular vocabulary.

Spell correction. The spell correction results in Tables 4, 5 and 6 show that spell correction behaves differently from clean word prediction. Although the Transformer is better for clean prediction, the N-gram model is competitive and sometimes better for spell correction, especially on TinyStories and WikiText-2. Our reasoning is that spell correction ranks whole-word candidates produced by edit distance. The N-gram model scores complete words directly, while the Transformer only uses the probability of the first BPE token as an approximation. This can work, but the issue is when several candidate words share the same first subword token. In spell correction, many candidates are already similar in spelling, so this loss of ranking precision matters more. However, for the Mobile SMS dataset, the Transformer once again beats the N-gram model. Our reasoning is that SMS messages often contain short repeated conversational patterns, such as common greetings, replies, and everyday phrases. The Transformer can learn these patterns even when the context is short. The N-gram model is more affected by the large and irregular SMS vocabulary, especially when spelling errors increase the number of possible candidates.

Effect of context length. When analyzing spell-correction results with respect to context length at top-3, we observed an interesting pattern. Table 7 shows that the Transformer benefits more from longer context, especially on TinyStories and Mobile SMS. For TinyStories under S1, the Transformer improves from 37.63% in the 1–2 context-word interval to 66.13% in the 21–50 interval. A similar pattern appears for Mobile SMS, where the Transformer reaches 66.07% under S1 in the longest context interval. The plot next to Table 7 also confirms this: the Transformer matches the N-gram model relatively quickly. After 11–20 context words, the Transformer outperforms the N-gram model. This comes back to the idea that the Transformer can use additional context better when the surrounding text is informative. The N-gram model is more limited because it only uses a fixed number of previous words as context, i.e., up to three previous words.

Spell-correction strategies. S1 is the strategy that gave the best results across all models and datasets. This is reasonable, since allowing only one edit keeps the candidate set small and easier to rank. S2 allows more corrections, but adds more incorrect candidates and therefore lowers performance. Interestingly, S3 performs almost the same as S2, which means that setting the transposition edit cost to 0.5 did not provide a clear advantage in our evaluation.

Limitations.

The evaluation has four main limitations. First, the N-gram and Transformer models were tested on different pre-processing formats and test-set sizes, so the results are a practical system comparison rather than a fully controlled one. Second, Transformer word scoring uses only the first BPE-token probability, which can lose ranking precision, especially in spell correction. Third, spell errors were synthetic, so they may not reflect real typing-error distributions. Finally, N-gram perplexity is word-based while Transformer perplexity is BPE-token-based, so the absolute values are not directly comparable.

7 Conclusion

In this project, we implemented and evaluated spell-aware word prediction using an interpolated N-gram model and a Transformer language model across three different text domains. The Transformer achieved the best results for clean word prediction. However, the N-gram model remained a strong baseline for spell correction, especially on TinyStories and WikiText-2, because it scores whole-word candidates directly and works well with the edit-distance candidate generation. For longer contexts, however, the Transformer matches and even outperforms the N-gram model on spell correction.

The results also show that dataset characteristics matter. TinyStories benefits from longer local N-gram context, WikiText-2 is more challenging due to its varied vocabulary and syntax, and Mobile SMS favours the Transformer in both clean prediction and spell correction. Among the spell-correction strategies, S1 gave the best results.

References

- [1] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Berlin, Germany: Association for Computational Linguistics, 2016, pp. 1715–1725.
- [2] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [3] F. J. Damerau, “A technique for computer detection and correction of spelling errors,” *Communications of the ACM*, vol. 7, no. 3, pp. 171–176, 1964. DOI: 10.1145/363958.363994.
- [4] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.
- [5] F. Jelinek and R. L. Mercer, “Interpolated estimation of Markov source parameters from sparse data,” in *Proceedings of the Workshop on Pattern Recognition in Practice*, E. S. Gelsema and L. N. Kanal, Eds., Amsterdam: North Holland, 1980, pp. 381–397.
- [6] R. Kneser and H. Ney, “Improved backing-off for M-gram language modeling,” in *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing*, vol. 1, 1995, pp. 181–184. DOI: 10.1109/ICASSP.1995.479394.
- [7] M. D. Kernighan, K. W. Church, and W. A. Gale, “A spelling correction program based on a noisy channel model,” in *COLING 1990 Volume 2: Papers Presented to the 13th International Conference on Computational Linguistics*, 1990.
- [8] R. Eldan and Y. Li, *TinyStories: How small can language models be and still speak coherent English?* 2023. arXiv: 2305.07759 [cs.CL].
- [9] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” in *International Conference on Learning Representations*, 2017.
- [10] K. Vertanen and P. O. Kristensson, *Mobile text dataset*, Mobile Text Dataset and Language Models, Dataset 1, 2019. [Online]. Available: <https://digitalcommons.mtu.edu/mobiletext/1/>.
- [11] S. F. Chen and J. T. Goodman, “An empirical study of smoothing techniques for language modeling,” *Computer Speech and Language*, vol. 13, no. 4, pp. 359–393, 1999.
- [12] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019.

8 Appendix

Laplace Smoothing

Laplace smoothing adds one pseudo-count to every possible continuation of a context c . For a word $w \in V$, the smoothed conditional probability is

$$P_{\text{Laplace}}(w | c) = \frac{\text{count}(c, w) + 1}{\text{count}(c) + |V|}. \quad (4)$$

Here, $\text{count}(c, w)$ is the number of times w follows context c , $\text{count}(c)$ is the total number of times c appears as a context, and $|V|$ is the vocabulary size. The added $|V|$ in the denominator accounts for adding one pseudo-count to each word in the vocabulary, ensuring that every $w \in V$ receives non-zero probability.

Perplexity

$$\text{PPL} = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(x_i | x_{<i}) \right). \quad (5)$$

Here, x_i is the next word or token. Lower perplexity means the model assigns higher probability to the held-out sequence. For the N-gram model, we computed it over words; for the Transformer, we computed it over BPE tokens.

Distribution of edit operations

Table 8: Distribution of edit operations in the corrupted spell-correction test sets. Del. = deletion, Ins. = insertion, Sub. = substitution and Trans. = transposition.

Dataset	Strat.	Total ops	Del.	Ins.	Sub.	Trans.
TinyStories	S1	999	171 (17.1%)	289 (28.9%)	275 (27.5%)	264 (26.4%)
	S2	1,751	343 (19.6%)	462 (26.4%)	494 (28.2%)	452 (25.8%)
	S3	1,751	343 (19.6%)	462 (26.4%)	494 (28.2%)	452 (25.8%)
WikiText-2	S1	902	204 (22.6%)	243 (26.9%)	227 (25.2%)	228 (25.3%)
	S2	1,564	357 (22.8%)	402 (25.7%)	436 (27.9%)	369 (23.6%)
	S3	1,564	357 (22.8%)	402 (25.7%)	436 (27.9%)	369 (23.6%)
Mobile SMS	S1	98	13 (13.3%)	32 (32.7%)	25 (25.5%)	28 (28.6%)
	S2	168	25 (14.9%)	49 (29.2%)	53 (31.5%)	41 (24.4%)
	S3	168	25 (14.9%)	49 (29.2%)	53 (31.5%)	41 (24.4%)

Distribution of context length before the corrupted word by dataset

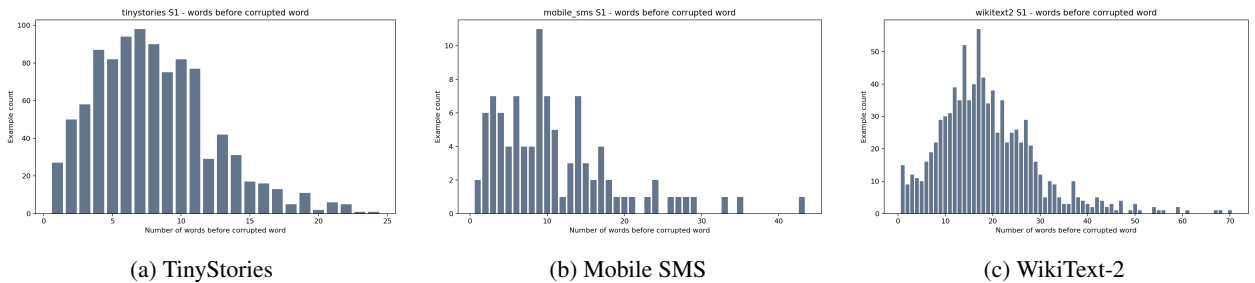


Figure 2: Distribution of context length before the corrupted word by dataset.

Spell-parameter weights

Table 9: Best spell-ranking parameters found on validation examples using strategy S3.

Dataset	Model	Validation examples	λ_{edit}	μ_{freq}	Top-3 Acc.	Top-3 KS
TinyStories	N-gram	100	1.0	0.05	90.0%	51.9%
WikiText-2	N-gram	100	1.0	0.20	71.9%	27.3%
Mobile SMS	N-gram	20	1.0	0.05	90.0%	32.5%
TinyStories	Transformer	100	1.0	0.05	86.0%	49.0%
WikiText-2	Transformer	20	1.0	0.05	57.9%	23.0%
Mobile SMS	Transformer	20	1.0	0.05	85.0%	34.0%

The validation results show that $\lambda_{\text{edit}} = 1.0$ was selected in all settings, so edit distance was important for candidate ranking. The frequency weight was usually $\mu_{\text{freq}} = 0.05$, except for the WikiText-2 N-gram model, where $\mu_{\text{freq}} = 0.20$ performed best. The N-gram model performed better on TinyStories and WikiText-2, while Mobile SMS was mixed: the N-gram model had higher accuracy, but the Transformer had the higher KS-ratio.

Transformer grid-search results

Table 10: Transformer grid-search results. Acc. = top-k accuracy; KS = saved-keystroke ratio.

Dataset	Arch.	Stride	Blocks	Heads	Dim.	Params	Val. loss	Top-1 Acc.	Top-2 Acc.	Top-3 Acc.	Top-4 Acc.	KS
WikiText-2	default	8	4	4	256	5,851,016	3.8610	84.51%	88.04%	89.49%	90.29%	37.60%
WikiText-2	medium	8	6	8	512	24,289,160	3.8654	83.89%	87.44%	88.90%	89.79%	37.66%
WikiText-2	medium	64	6	8	512	24,289,160	3.9469	83.73%	87.51%	88.91%	89.81%	37.37%
WikiText-2	default	32	4	4	256	5,851,016	4.1071	83.67%	87.29%	88.82%	89.68%	37.16%
WikiText-2	medium	32	6	8	512	24,289,160	3.9386	83.35%	87.08%	88.61%	89.48%	37.49%
WikiText-2	default	64	4	4	256	5,851,016	4.4868	83.28%	87.02%	88.77%	89.79%	35.39%
WikiText-2	small	8	2	2	128	1,746,056	4.4918	83.17%	87.09%	88.63%	89.55%	35.74%
WikiText-2	small	32	2	2	128	1,746,056	4.6856	83.00%	86.94%	88.56%	89.50%	34.23%
WikiText-2	small	64	2	2	128	1,746,056	4.9388	82.30%	86.50%	88.41%	89.46%	33.42%
Mobile SMS	default	32	4	4	256	5,752,712	4.8376	90.68%	95.76%	96.61%	96.61%	40.60%
Mobile SMS	default	8	4	4	256	5,752,712	4.8402	89.83%	94.49%	96.61%	96.61%	42.37%
Mobile SMS	small	32	2	2	128	1,696,904	5.1143	88.98%	94.49%	96.19%	96.61%	38.94%
Mobile SMS	small	8	2	2	128	1,696,904	5.1159	88.56%	94.07%	96.19%	96.61%	38.11%

N-gram vocabulary size by dataset

Table 11: N-gram vocabulary size by dataset.

Dataset	N-gram vocabulary size
TinyStories	33,704
WikiText-2	64,758
Mobile SMS	148,938